

Corporate Expense & Approval System

Technical Report

Case organisation: DeloitteEdge Consulting Ltd (fictional)

Name: Abdul Mateen

Student ID: 20093908

Programme: MSc in Information Systems and Computing

Module: B9IS130 – Web Development for Information Systems

Lecturer: Dr. Obinna Izima

Date: 2 July 2026

Live application: <http://testmateen.norwayeast.cloudapp.azure.com:8080/>

Source code:

<https://github.com/AbdulMateenzwl/B9IS123-Programming-for-Information-Systems> (branch:
project)

Contents

1	Introduction	3
2	Research Process and Requirements Analysis	3
2.1	Problem domain research	3
2.2	Stakeholders and requirements	4
2.3	Feasibility and scope decisions	4
3	System Architecture	5
3.1	Overall structure	5
3.2	A modular monolith, not microservices	5
3.3	Frontend architecture	5
3.4	Data model	6
4	Technology Choices	6
4.1	Angular over React	6
4.2	NestJS over plain Express	6
4.3	MongoDB over PostgreSQL	7
4.4	Supporting choices	7
5	System Features	8
5.1	Authentication and Role-Based Access Control	8
5.2	Expense Claim Management	8
5.3	Expense Item Management	9
5.4	Multi-Step Approval Workflow	9
5.5	Budget Management and Enforcement	10
5.6	Role-Aware Dashboards	10
5.7	Reporting and Analytics	10
5.8	File Attachments	11
5.9	User and Department Administration	11
5.10	Validation and Error Handling	11
5.11	Supporting Development Features	12
6	Security Implementation	12

6.1	Authentication and session management	12
6.2	Authorisation	12
6.3	Input validation and injection defence	12
6.4	Abuse prevention and remaining gaps	13
7	Testing Strategy	13
8	Deployment Process	14
8.1	Containerised architecture	14
8.2	Cloud platform decision	14
8.3	Deployment evaluation	15
9	Critical Evaluation and Future Work	15
10	Conclusion	16
11	Acknowledgement of AI and External Resources	16

1 Introduction

This report critically examines the design, implementation and deployment of the newly developed Corporate Expense & Approval System for fictional mid-sized professional services firm, DeloitteEdge Consulting Ltd. The new information system has replaced the existing corporate process for managing corporate expenses which until recently were dealt with by using spreadsheets within a spread sheet. Employee submitted expense claims are now entered within the system by the individual. Subsequently, submitted claims are reviewed and approved in a multi-step process by the individual's line manager and/or by the firm's finance officer(s). Administrator(s) manage system data including user information, individual's department(s) and allocated expenditure budgets. A series of role-specific dashboards have been provided that allow claimers to check the status of their submitted claims. approve/dispense payments, managers to monitor the claims from their reports and check details of individual submitted claims and finance officers to check departmental spending by category and by department.

The application is built as a full-stack JavaScript solution, based on the MEAN stack of MongoDB for data storage, a NestJS REST API (based on Node.js/Express), and an Angular-based Single-Page-Application for front-end functionality. The solution is containerized and deployed to a public facing Microsoft Azure VM. This report is not just a description of the implemented system. Every part of the report has a critical twist; decisions have been made and alternatives have been rejected. Trade-offs have been accepted. A critical evaluation is performed of the research conducted, the requirements analysis, the implemented system architecture, the selected technology, the security implementation, the testing strategy, the used deployment process, and finally the generated and implemented system, highlighting its limitations and pointing out directions for future work.

2 Research Process and Requirements Analysis

2.1 Problem domain research

I began the project by investigating how organizations currently handle employees' expenses, and how exactly manual expense handling can go wrong. Information systems for transaction processing are typically designed to support a centralised processing system, high auditability and sufficient management reporting for operational tasks. A simple example of a system which fails in all these areas is a manual expense processing system, with no logs of when or who approved expenses going through various stages of an employee's claimed expenses could get duplicated and never processed or even discarded at some point. Also finance employees would not have adequate information for making decisions regarding spending within a particular budget and year-end processing would be a laborious, prone-to-error task.

The key features of current systems, which users have come to expect, were gathered from a variety of commercial systems, including SAP Concur and Expensify. These include a claim going through a number of states, an approval process that can be configured into a number of steps and finance staff able to view budgets. The key features of current systems, which users have come to expect, were gathered from a variety of commercial systems, including SAP Concur

and Expensify. These include a claim going through a number of states, an approval process that can be configured into a number of steps and finance staff able to view budgets. The main features of the system were then used to form the basis of the requirements for the system. As the main aim of the assessment was for the student to learn how to implement a full-stack system from scratch for a case study organization, a bespoke system was the best option as it would allow the approval process to be modeled exactly as for the two-step process (approve then submit to finance) for the case study organization as opposed to restricted by commercial licensing.

2.2 Stakeholders and requirements

Within the proposed system there are four main stakeholder roles. These roles will have their own goals and the main functional requirements for the system will come from these roles. Firstly, there is the employee who will submit a claim and be able to track its progress. The line manager will approve the first stage of the claim for their team. The finance officer will approve the claim at the final stage and will also oversee the budget. Lastly, there is the system administrator who will manage the users, departments and budgets and be able to make user, department and budget management changes within the system.

The non-functional requirements for this assessment were security (as per OWASP Foundation (2021)) and usability. This means that the system must work on any device with a decent screen size and that the user experience needs to be good enough. In terms of design, there must be a number of features to support a responsive design, which is to say that the system must work on both desktop and mobile devices. To support maintainability the system must be designed to be modular. Finally, in order to support reproducible deployment the system must be designed to be deployable to different environments with minimal changes.

2.3 Feasibility and scope decisions

The requirements have been prioritized with the aid of analysis so as to keep the assessment within the timeframe. The following ‘could-have’ features have been descoped as they introduced external dependencies which would take too much time to implement and to handle failures for the assessment: OCR receipt scanning, email notifications to employees and line managers after a claim has been submitted, and multi-currency conversion for claims. Note that all of these have been descoped for reasons related to the external dependencies and thus the time that would be needed to integrate them as well as to handle any failures that may occur. These are therefore revisited in Section 9. For example, the currency field on the claims screen is currently displayed as information only and thus no work has been undertaken to process this field.

3 System Architecture

3.1 Overall structure

A Three Tier Client Server System, where the Angular Single Page Application (Presentation Tier) talks to the NestJS REST API (Application Tier) which in turn stores and retrieves data from MongoDB (Data Tier). Communication between the Client and the Server is done via HTTP Protocol with JSON payloads following the fundamental principles of REST Architecture such as stateless and uniform resource-based interface. Given that the server does not hold any session state, it is very easy to scale the application tier to scale horizontally behind a load balancer, without having to worry about session affinity, even though in this deployment, we are only running a single instance of the server.

3.2 A modular monolith, not microservices

An early design choice was to choose between a monolithic modular application and a set of microservices. Given the scope of this project, it would introduce too many operational concerns for the current scale of the application. In the end, I settled on building a modular monolithic application which I have split into 9 NestJS feature modules. This will enable me to split out individual feature modules into their own services should required in the future (i.e. growth etc). Currently, however, it is too early to consider such a change and thus all features are in a single monolithic backend application. The 9 modules are: auth, users, departments, budgets, claims, items, attachments, workflow and dashboard. Each feature module is independent of the others with each having its own controllers, services, domain models (Mongoose schema's) and Data Transfer Objects (DTO's) that are specific to that feature.

The common layer contains cross cutting concerns of the application, such as JWT and role guards, the per-user rate limiting guard, custom decorators and the global exception filter which normalizes all exceptions to a JSON envelope. The request handling is layered, so the business rules of a feature are testable without the HTTP concerns of the controller.

3.3 Frontend architecture

The Angular (2026) frontend follows a similar pattern of modular construction. The core layer contains services for interactions with the API, including typed versions for easier use. Also in the core layer is an authentication service for handling logins and logouts, a route guard to ensure that only logged in users can view protected routes, a JWT HTTP interceptor to add a valid JWT to every HTTP request, and a series of custom decorators for use in components. The global exception filter is also in the core layer. Every page in the application is a single component, and as each page is for a feature, there is a page for the dashboard, for claims, for viewing items, for workflow, for users, for departments, and for budgets. The dashboard page contains several Charts.js based charts, a doughnut chart showing the status of all claims by category, a doughnut chart made using library Chart.js (2026) showing total spend by category, and a stacked bar chart for the current budget for each department, showing actual spend against allocated for each category.

3.4 Data model

The system uses seven collections to store its data: users, departments, budgets, claims, items, workflows and attachments. In terms of data model design, one of the major decisions made in the design of this system was whether to use referencing or embedding for the Expense Items of claims. Generally in MongoDB, items that are part of other documents are embedded within the parent document. For example, in the case of a Claim, the Items would be embedded within the Claim. However, in this system, the Expense Items are stored in their own collection with a reference to the Claim to which they belong. This was a deliberate design decision as it allows the application to handle the CRUD (Create, Read, Update, Delete) of the Items for a Claim while the Claim is in the Drafts state. In addition, the spend for Items of a particular Category can be easily aggregated across all Claims for that Category using the aggregation pipeline on the items collection. The trade-off of this design is that the Total of a Claim and its Items can only be kept up to date on an application level, as the system does not support multi-document transactions, which is a common limitation in using a document store for financial data and is discussed further in Section 4.3.

4 Technology Choices

This section outlines the reasons behind the choice of technology to meet the requirements of the brief. The brief is JavaScript-based so the main decision was which JavaScript-based technologies to use for the front-end and back-end. For the front-end Angular was chosen over React for the reasons stated above. For the back-end NestJS was chosen over Express for similar reasons to the front-end, a structured approach to back-end development removes the need for integration decisions and reduces risk and inconsistency. In addition, the accepted costs of a steeper learning curve and larger initial bundles are acceptable for this module as they can be lazy loaded as earlier.

4.1 Angular over React

This section outlines the reasons behind the choice of Angular over react. I have choose Angular due to its vast builtin functions instead of relying on seperate libraries. React uses external libraries for almost everything this causes issues when there is version mismatch for dependencies, where this kind of things does not happen in Angular. So Angular provides a much stable output.

4.2 NestJS over plain Express

For the backend, I chose to use NestJS (2026) as the framework over Express as it adds a layer of structure on top of Express. Express is a minimalistic and unopinionated framework which leaves matters such as structure, validation and dependency management to the developer. As previously stated, this leads to the issue of architectural drift. NestJS, on the other hand, provides a framework that supports an architecture of modules with dependency injection. This is provided using a set of decorators which also include support for guards, pipes and interceptors. This means that as previously stated, there is support for a structured architecture. It also means

that the overall architecture of the backend is very similar to the frontend with both being based on an Angular-like framework and both written in TypeScript. This greatly reduces the impact of context-switching when moving between the two, as there is less to learn. The major drawback is the added complexity of the framework as well as potential lock-in. However, this is greatly mitigated by the fact that the core of the HTTP handling is still provided by Express.

4.3 MongoDB over PostgreSQL

MongoDB Inc. (2026) rather than PostgreSQL for database. Relational databases (such as PostgreSQL) are typically used for financial systems that require ACID transactions (i.e. guaranteed consistency of updates) and referential integrity. MongoDB would have been a less obvious choice in this case. In the end, the claim (i.e. a financial assertion, such as a customer's invoice or credit note) and the related items, all attachments and all the different workflow steps were all going to be represented as documents in the database. Moreover, the nature of the aggregation for the dashboard queries, would have been very easily handled by using the aggregation pipeline in MongoDB (as compared with, for example, the grouping and sum functions in SQL). In terms of schema flexibility (i.e. how easy would it have been to modify the design of the database, as requirements developed on the financial assertion module), MongoDB was also felt to be far better to PostgreSQL. That is not to say that there is no integrity risk with using MongoDB, rather than PostgreSQL. The integrity of the data (i.e. the accuracy and consistency of the stored data) could still potentially be compromised by the way in which the data is stored in the MongoDB documents. As stated earlier, the structure of the Mongoose schema for each of the different types of documents, would enforce the structure of the data for each of the different types of claims (i.e. financial assertions), as well as the different types of items and the different types of attachments. And, in addition, it would be written on a per document basis, as far as possible, so that there would not be a requirement for a series of updates to a single document in order to implement a single transaction (i.e. to add a single new item to a claim). In terms of monetary values, as previously stated, these would be aggregated from the underlying source records. A financial system (i.e. a system that processes financial transactions and manages associated data) would require the use of PostgreSQL, rather than MongoDB, in order to guarantee stronger consistency for production transactions.

4.4 Supporting choices

Chart.js was chosen instead of D3.js because D3.js gives you unlimited flexibility in how you create your charts, which for the simple standard charts in this project would mean building each of the individual primitives for each chart type. Chart.js, on the other hand, provides a set of simple charts (including line, bar, pie and donut charts) out of the box, all of which can be customized to meet your needs, but all of which can be used as is for simple charts. Docker and Docker Compose were used to package up the backend and to run the backend and the frontend in separate containers so that the developer can guarantee that the exact same environment is running in development as is running in production. TypeScript was used for the backend as well as the frontend because API responses from the backend functions are type-checked as soon as they are typed and consumed by the frontend code. This means that any contract-dirt (i.e.

where the backend function’s API contract and the frontend code’s consumption of that contract differ) is caught at compile-time rather than at runtime.

5 System Features

The application delivers an end-to-end corporate expense management workflow, replacing an ad hoc, spreadsheet-driven process with a centralised information system. Functionality is organised around a single-page Angular client that communicates with a modular NestJS REST API, with all persistence handled in MongoDB. The features described below span the full claim lifecycle—from submission through multi-step approval to budget reconciliation and reporting—while enforcing role-appropriate access at every layer.

5.1 Authentication and Role-Based Access Control

Access to the system is governed by stateless, token-based authentication. Users authenticate with an email and password, after which the API issues a signed JSON Web Token (JWT) together with a longer-lived refresh token. The access token carries the user’s identity and role in its payload, allowing authorisation decisions to be made without repeated database lookups.

- Passwords are hashed with `bcrypt` and never stored in plain text; the `passwordHash` field is stripped from every API response via a Mongoose `toJSON` transform.
- Access tokens expire after 8 hours; a refresh token (valid for 7 days) allows the session to be renewed without re-authentication.
- Four distinct roles are supported—Employee, Manager, Finance Officer, and Admin—each mapped to a specific set of permissions.
- Authorisation is enforced at two levels: route protection on the server via a NestJS `RolesGuard`, and a role-aware client interface that conditionally renders navigation items, pages, and action buttons.

The permissions associated with each role are summarised in Table 1.

Table 1: Role-based permission overview.

Role	Primary Permissions
Employee	Submit, edit, and track own expense claims
Manager	Configure approval steps; approve or reject claims
Finance Officer	Monitor budget utilisation and departmental spend
Admin	Full system administration of users, departments, and budgets

5.2 Expense Claim Management

Expense claims are the central entity of the system and support full CRUD operations, constrained by the claim’s current status. Each claim progresses through a defined lifecycle—*Draft*,

Submitted, *Under Review*, *Approved*, and *Rejected*—and the actions available to a user depend on both this status and their role.

- Employees create claims in a *Draft* state and may edit their description and currency while the claim remains a draft.
- Claims can be filtered by status, with employees seeing only their own claims and managers and administrators seeing all claims.
- Submission is validated at the service layer: a claim must contain at least one expense item before it can be submitted for approval.
- Deleting a draft claim cascades to its related items, workflow steps, and attachments, preserving referential consistency.

5.3 Expense Item Management

Individual expenditure lines are recorded as expense items attached to a draft claim. Each item captures a category, amount, date, and description, and the parent claim's total is maintained automatically.

- Items are classified into one of eight categories: Travel, Accommodation, Meals, Equipment, Training, Subscriptions, Entertainment, and Other.
- The claim's `totalAmount` is recalculated on every item add, edit, or delete, ensuring the displayed total is always consistent with the underlying line items.
- Item management is restricted to claims in the *Draft* state, preventing modification of expenditure after submission.

5.4 Multi-Step Approval Workflow

A structured, sequential approval workflow replaces the informal sign-off of the manual process and provides a clear audit trail for every decision.

- A manager assigns a minimum of two sequential approvers to a submitted claim.
- Approval steps are enforced strictly in order: a later step cannot be actioned until the preceding step has been completed.
- Each approver may approve or reject a claim, optionally attaching a comment to explain the decision.
- A rejection at any step immediately halts the workflow and marks the claim as *Rejected*, notifying the workflow of the outcome.
- To preserve integrity, an employee cannot approve their own claim.
- Final approval triggers an automatic budget check against the relevant department's allocation.

5.5 Budget Management and Enforcement

The system tracks departmental budgets on a per-fiscal-year basis and reconciles them automatically against approved expenditure, addressing the overspending and lack of oversight present in the original process.

- Administrators create an annual budget for each department and fiscal year, and may update the allocated amount.
- Budgets can be filtered by fiscal year, and utilisation is surfaced against each department's allocation.
- On final approval of a claim, the corresponding department budget is decremented automatically, keeping actual spend and allocation in sync.

5.6 Role-Aware Dashboards

Each role is presented with a tailored dashboard that summarises the information most relevant to that user, loaded automatically on login based on the authenticated user's role.

- **Employee:** personal claim summary and approved spend by category.
- **Manager:** count of pending approvals and recent submissions requiring attention.
- **Finance Officer:** budget utilisation per department, presented with progress indicators.
- **Admin:** system-wide statistics including user counts by role and the organisation's top spenders.

5.7 Reporting and Analytics

Beyond operational dashboards, the system provides consolidated reporting to support financial oversight and year-end review.

- Spending by department, comparing budgeted allocation against actual approved expenditure.
- Spending by employee, ranked by total approved amount.
- Spending by expense category, including per-category item counts.
- Monthly trends showing approved, rejected, and pending claims over time.
- A fiscal-year selector to switch the reporting period.

5.8 File Attachments

To support verification of expenditure, users can attach supporting documentation—such as receipts—directly to a claim.

- Supported formats are PDF, JPG, JPEG, and PNG, with a per-file size limit of 10 MB enforced on upload.
- Uploads are handled server-side via Multer middleware and stored on the application server.
- Removing an attachment deletes the corresponding file from disk as well as its database record, preventing orphaned files.

5.9 User and Department Administration

Administrative features allow the organisational structure and its users to be maintained without direct database access.

- Administrators create users with an assigned role and department, and may update user details, job title, and role.
- Users are deactivated through a soft delete, which disables access while preserving historical data and audit integrity.
- Departments can be created and updated with a name, location, and assigned manager, and are likewise deactivated rather than hard-deleted.

5.10 Validation and Error Handling

Data integrity is protected through complementary validation at both tiers of the application, backed by consistent error reporting.

- Server-side validation is applied to all incoming request bodies using `class-validator` decorators on Data Transfer Objects, with a global validation pipe configured in whitelist mode to reject unexpected fields.
- Fifteen documented business rules are enforced at the service layer, ensuring that domain constraints (such as ownership and status transitions) hold regardless of client behaviour.
- Client-side checks validate required fields before requests are dispatched, reducing unnecessary round trips.
- A global exception filter returns errors in a consistent JSON structure, giving the frontend descriptive, uniform feedback on failure.

5.11 Supporting Development Features

Two features support development, demonstration, and quality assurance rather than end-user workflows.

- **Seeded demo data:** a seed script populates the database with 6 departments, 12 users, and 6 budgets, along with claims in every status and pre-configured workflow steps, enabling immediate end-to-end demonstration.
- **API testing collection:** a Bruno collection of 47 requests covers all endpoints, with environment variables and post-response scripts that automatically capture the JWT and entity identifiers to streamline manual testing.

6 Security Implementation

For each of the security controls selected for this system, the author worked through the OWASP Top 10 (OWASP Foundation, 2021) and determined which of the risks applied to this particular system. The result is a defense in depth approach to securing the authentication, the authorisation, the input and the transport level headers for the API.

6.1 Authentication and session management

The authentication of users is handled by JSON Web Tokens. The login function issues a short-lived access token (8 hours) and a longer-lived refresh token (7 days). This allows the application to remain stateless. This is a known trade-off for stateless tokens, they cannot be revoked before their expiry time. But the short access-token lifetime bounds this exposure window. The refresh endpoint can be used to rotate the tokens. The passwords are stored hashed with bcrypt (using bcrypt with work factor 12). Bcrypt is designed with an adaptive cost, this means the hashing function can get more expensive as hardware capability increases, unlike for example SHA-256 a fast general-purpose hash function, which is not suitable for password storage.

6.2 Authorisation

The authorisation is done by making sure the correct roles are added to a user, and that these roles are checked server side, using a declarative way of decorating controllers, services etc. Using a Angular route guard and enabling navigation for certain roles based on the current users roles, makes for a better user experience. But for the sake of security, these client side checks are treated as convenience and can be easily bypassed, so from a security point of view, the API will treat every request as hostile.

6.3 Input validation and injection defence

NestJS applications can be globally configured to include a ValidationPipe which specifies the behavior for any unknown properties in request bodies. In this application, the ValidationPipe is

set to run with the ValidationPipe options of whitelist and forbidNonWhitelisted enabled. This configuration ensures that any unknown properties will be rejected, thereby preventing mass-assignment attacks. The data-transfer objects have been type-checked and, where applicable, their payload fields have been constrained by class-validator rules. NoSQL injection attacks are not possible as Mongoose parameterises queries rather than interpolating input. In terms of client-side scripting, Angular automatically encodes output values that have been interpolated within the HTML template, whilst on the server, all API responses are wrapped with Helmet security headers, including the content security policy (CSP) as well as the X-Content-Type-Options header. File uploads are limited to specific MIME types as well as file size, via a Multer filter, to prevent the upload of executable content that is stored as a receipt.

6.4 Abuse prevention and remaining gaps

Rate limiting can be put in place at a global level using the global throttler. Additionally, per-user rate limiting can be put in place in order to further limit the number of requests from a particular user. Helmet (2026) library is used to make implement Rate Limiting functionality. There is also the matter of CORS which is currently set to allow requests from any origin. However, an allowlist of origins could be set in order to further restrict access to the application. Of the areas currently left to be secured the biggest area for concern would be the application is currently deployed to serve over HTTP rather than HTTPS. In this case all tokens and credentials would be transmitted in cleartext which would defeat the purpose of using a secured token in the first place. Furthermore, although all areas of the application currently affected by XSS have been secured to prevent such an attack in the future, storing JSON Web Tokens in browser storage (localStorage in this case) is not as secure as storing them in HttpOnly cookies with CSRF protection. The final area that would be secured is account lockout and multi-factor authentication. All of these security issues would be addressed in future work with the biggest area of concern (securing the application to serve over HTTPS) being the first to be addressed.

7 Testing Strategy

The testing strategy was risk-based as there was a limited amount of time to identify and fix defects. Therefore, the focus was placed where the most damage would occur. The two areas that were deemed to be highest-risk were the authentication (a security boundary) and the claim lifecycle (the core business process of the service). Both of these areas are covered by the Jest unit test suites for the service layer. These test suites use mocked out Mongoose models to test the service layer in isolation. The password verification and token issuance for authentication, and the creation, submission, status transition, and the ownership rules for claims are all tested.

In addition to the unit tests, I performed systematic testing of the API interfaces. A script was written to seed a repeatable test dataset (6 departments, 12 users (all roles), budgets for all users, and a number of claims in all states). I then worked through all of the endpoints, creating authenticated HTTP requests for each of the 4 roles, and verified both correct functionality and the authorization errors that should occur. The test data also enabled me to verify the role specific dashboards, which also acted as user acceptance tests for these interfaces. One defect

was discovered during this process, a seeded claim had a fixed prior year submission date, so when the aggregation functions in the dashboard views were filtered to the current year, the results for that year were empty, resulting in the associated charts being blank. This proved that using a seed of data was an effective method for testing, as the defect would not have been found by just running the unit tests, as the code for the aggregation functions was correct.

The proposed test strategy for this project is honest in the sense that the test strategy describes the test cases that have been written rather than overstating test coverage. The test strategy, however, is incomplete in the sense that there are no automated integration tests and end-to-end tests for the system. In addition, there are no tests for modules outside the two critical modules for which test cases were defined. A production-bound version of this test strategy would therefore include supertest-based API tests as well as end-to-end tests for all possible user journeys for each role using something like Playwright. In addition, a GitHub Actions pipeline would be set up to test the application and fail the merge if the test suite does not 100% pass. The lack of a continuous integration process for the test suite is the biggest process gap for this project.

8 Deployment Process

8.1 Containerised architecture

At the end the project was deployed on the cloud. It consists of three containers: one for the MongoDB database with a persistent volume and healthchecks for the database itself, one for the NestJS backend, which is built from a Node.js baseimage, and one for the Angular app, which is a production build and is served from an NGINX server. All three containers are on the same isolated bridge network (so they can all communicate with each other). The NGINX server has a dual role in the architecture: it serves the static Angular files, and it reverse-proxies requests to /api to the corresponding container (i.e. the backend container). This has the nice side effect that in production the browser and the backend share the same origin, so there is no need for any CORS configuration on the server (and more importantly, the database is never exposed to the outside world, only to the server itself, within the Docker Inc. (2026) network). All configuration is done via environment variables, in accordance with twelve-factor app principles. This means that the images are environment-agnostic.

8.2 Cloud platform decision

To achieve this, we set up a virtual machine on Azure Microsoft (2026) Azure in the Norway East region. The virtual machine is a Infrastructure-as-a-Service (IaaS) VM, not a Platform-as-a-Service (PaaS) like Azure App Service. Other PaaS options we could have used are Render and Railway. But we chose IaaS for several reasons. The primary reason is that we have full control over the Docker runtime on the VM. This means that we can use the exact same Compose file that we use for development on our local machines for production on Azure. This also means that we can manage the environment configuration and the deployment of our application to the VM in exactly the same way as we do for development. This is what this module is trying to assess. If we were to use a PaaS then we would have to use the PaaS's way of managing environment

configuration and deployment, which is what this module is trying to assess. The other reason is that there are real costs to running an IaaS VM as opposed to a PaaS. For example, we have to manually patch the operating system. There is no autoscaling. There is no managed TLS. And there is a single point of failure, i.e. the single VM. In a production scenario, we would use a managed MongoDB managed as a Replica Set, i.e. MongoDB Atlas, rather than as a Docker container co-located with the application.

8.3 Deployment evaluation

The end-to-end test of the public URL, serving the app and full login and claim workflow against the live API worked fine. Two things to fix: 1. TLS via Let's Encrypt w/ auto-renewal via NGINX F5 Inc. (2026) (pretty straight forward with the reverse proxy we have already). 2. Continuous deployment w/ GitHub Actions: build images, run test suite, then deploy on merge (replaces the current manual pull/rebuild process that we have now, but is repeatable and protects against accidentally deploying an untested commit).

9 Critical Evaluation and Future Work

Our new expense system meets all of the requirements outlined for it at the start of the project. All four defined roles are able to complete their required tasks fully. The approved process for the case organization has been correctly implemented with a full audit trail for all decisions and corresponding comments and timestamps. Real time budget utilization figures are available. The new application is publicly available and can be used from any device with a web browser. The core of the system is an opinionated, fully modular, end-to-end TypeScript stack that has provided the project with the required consistency and good rate of progress.

There are several areas where the application falls short. The application is currently deployed without a transport layer security (TLS) certificate, and the site should not be used over the public Internet until a suitable certificate is obtained and the site re-deployed. In addition to fixing the security related issue mentioned above, the site also requires a lot of work to make it accessible to people with disabilities. The application currently has basic support for reduced motion, and there are some basic keyboard focus indicators. However, the ARIA support for the application is extremely basic, and a full screen-reader accessibility test against the WCAG 2.1 guidelines W3C (2018) has not been performed. The automated tests for the application currently only test two of the main modules in the system. In addition, the validation DTOs used to validate the write functionality of the application do not currently test all of the endpoints with the same level of scrutiny. Finally, there are no external web services consumed by the system, and the natural next feature would be to integrate with a currency exchange API to allow claims to be made in any currency and then automatically converted to GBP when the claim is processed, given that the application currently stores the currency for a claim in the data model.

10 Conclusion

The major aim of this project was to design and implement an information system that would replace an existing failure-prone manual process of handling of expenses. The system was required to be a secure, web-based, information-based system. The above aim was met by the design and the implementation of a working information system that is deployed on the internet and is based on a solid MEAN stack. More importantly, each of the major design decisions, including the choice of Angular over React for the front-end, of NestJS over Express for the back-end, of MongoDB over PostgreSQL for the database, of a modular monolith over microservices, of an IaaS VM over PaaS for the hosting, were made with full understanding of the alternative choices and the trade-offs involved. The known shortcomings of the system have been listed and prioritized, with most of them having clear paths for remediation. The end result is not only an expense system, but also a solid information system with a robust engineering argument for its design and its implementation.

11 Acknowledgement of AI and External Resources

In terms of AI support, I made use of a coding assistant. I used Anthropic's Claude via the Claude Code tool. Claude Code was used for code review, for helping with debugging (including identifying the seed data bug in the dashboard seed data which is described in Section 6), for helping with refactoring and making code more neat and tidy and for help with generating documentation including this report. All generated code and assistance generated for documentation has been reviewed, tested, adapted where necessary and fully understood by the author. Reference has been made to all of the used frameworks, libraries and documentation in the list of references below.

References

Angular (2026), 'Angular documentation', Available at: <https://angular.dev>. (Accessed: 2 July 2026).

Chart.js (2026), 'Chart.js documentation', Available at: <https://www.chartjs.org/docs/>. (Accessed: 2 July 2026).

Docker Inc. (2026), 'Docker documentation', Available at: <https://docs.docker.com>. (Accessed: 2 July 2026).

F5 Inc. (2026), 'Nginx documentation', Available at: <https://nginx.org/en/docs/>. (Accessed: 2 July 2026).

Helmet (2026), 'Helmet.js documentation', Available at: <https://helmetjs.github.io>. (Accessed: 2 July 2026).

Microsoft (2026), 'Azure virtual machines documentation', Available at: <https://learn.microsoft.com/azure/virtual-machines/>. (Accessed: 2 July 2026).

- MongoDB Inc. (2026), ‘Mongodb manual: Data modeling’, Available at: <https://www.mongodb.com/docs/manual/data-modeling/>. (Accessed: 2 July 2026).
- NestJS (2026), ‘Nestjs documentation’, Available at: <https://docs.nestjs.com>. (Accessed: 2 July 2026).
- OWASP Foundation (2021), ‘Owasp top 10:2021’, Available at: <https://owasp.org/Top10/>. (Accessed: 2 July 2026).
- W3C (2018), ‘Web content accessibility guidelines (wcag) 2.1’, Available at: <https://www.w3.org/TR/WCAG21/>. (Accessed: 2 July 2026).